DSU Faculty Research Intelligence System

Detailed Implementation Plan

Purpose: Detailed blueprint for implementing the DSU Faculty Research Intelligence System: master research data, ORCID/authorized Scopus integration, verification, evidence, dashboards, and separate NAAC/NBA reporting.

1. Requirement Freeze

Before coding, finalize exactly what the system must collect, who can access it, what external sources are permitted, what evidence is required, and what reports must be produced. This prevents later redesign of the database and workflows.

Requirements to finalize

• Faculty master information and department structure.

• Journal, conference, book, chapter, patent, project, funding and other required research categories.

• ORCID iD and Scopus Author ID fields.

• Existing DSU research spreadsheets/databases for migration.

• User roles and approval permissions.

• Evidence/document requirements.

• Applicable current NAAC framework/manual.

• Applicable current NBA framework/manual/SAR/evaluation requirements for the relevant DSU program.

• Report scopes: faculty, department, institution and accreditation.

• Data retention, backup, security and audit requirements.

2. Official NAAC & NBA Requirement Mapping

The accreditation layer must be based on the current official documents applicable to DSU. Do not hard-code an old generic format. Store framework versions so requirements can change without changing the master research database.

Process: identify applicable documents → extract criteria/metrics → identify required fields → identify evidence → map database sources → define calculations → map report outputs → version the configuration.

Architecture rule: NAAC and NBA are separate reporting engines, but both consume the same verified master research data.

3. System Architecture

```
Faculty / Admin ↓ Web Frontend ↓ Backend API + Authentication ↓ Faculty + Research Management ↓ ORCID | Scopus/Elsevier API | Excel/CSV ↓ Staging → Normalize → Match → Duplicate Check → Validate ↓ Verification Queue ↓ MASTER RESEARCH DATABASE ↓ NAAC Engine | NBA Engine ↓ Dashboards / PDFs / Excel / Reports
```

The architecture separates external integrations from the master database. External data first enters staging; only approved information becomes official DSU data.

4. Database Design

Faculty and organization: departments, faculty, users, roles, permissions, research_identifiers.

Research: publications, publication_authors, journals, conferences, books, book_chapters, patents, research_projects, funding_records, research_activities.

Operations: sync_jobs, sync_records, verification_records, evidence_documents, audit_logs.

Accreditation: frameworks, framework_versions, criteria, subcriteria/metrics, criteria_mappings, evidence_requirements, report_templates, generated_reports.

Faculty: faculty_id, employee_id, name, email, department_id, designation, ORCID, Scopus Author ID, status, timestamps.

Publication: publication_id, title, DOI, publication type, journal, publisher, ISSN/eISSN, volume, issue, pages, date, source, external ID, verification status.

Author relationship: publication_id, faculty/external author ID, author order, affiliation and matching confidence.

5. User Roles

Role	Detailed responsibility
Faculty	View profile; confirm identifiers; review imported research; add/correct permitted data; upload evidence; track verification.
Department Admin	Manage assigned department faculty; review/verify research; monitor missing data; create department reports.
Research/Accreditation Admin	Institution-wide research, synchronization, verification, evidence, accreditation configuration and reports.
Super Admin	Users, roles, system settings, integrations, framework versions, security and audit.
Viewer/Management	Read-only dashboards and approved reports.

6. Faculty Management Module

• Faculty list, search and filters.

• Create/import faculty records.

• Department and designation assignment.

• Employee/institutional ID.

• ORCID and Scopus Author ID.

• Profile completeness.

• Active/inactive status.

• Linked research summary.

• Access-controlled editing.

7. Research Management Module

• Journal publications.

• Conference publications.

• Books and book chapters.

• Patents.

• Research projects.

• Sponsored research/funding.

• Research activities/courses where required.

• Citation/scholarly metrics where permitted by the source.

• Evidence attachment.

8. Faculty Research Profile

```
Faculty Name | Department | Designation ORCID | Scopus Author ID Publications | Conferences | Books |
Patents | Projects | Funding Research by Year Evidence Verification Status Last Synchronization / Last
Updated
```

The profile must clearly separate verified, pending, rejected and corrected records.

9. Manual / Excel / CSV Import

• Upload file and preview rows.

• Map columns to system fields.

• Validate required fields and formats.

• Show row-level errors.

• Create staging records.

• Run matching and duplicate checks.

• Send valid records to verification.

• Never silently overwrite approved master records.

10. ORCID Integration

ORCID is used as a researcher identifier and authorized data source. The system should use official API access rather than scraping public pages.

• Faculty confirms ORCID iD.

• Validate identifier.

• Authorize API access as applicable.

• Retrieve permitted researcher/work information.

• Store results in staging.

• Match works to faculty.

• Check duplicates.

• Verify and approve.

• Store sync status, timestamp and errors.

11. Scopus / Elsevier Integration

Scopus integration requires DSU to have appropriate authorized Elsevier/Scopus API access. A normal website account should not be assumed to provide API access.

• Confirm DSU institutional subscription/API entitlement.

• Obtain authorized API credentials.

• Store credentials only on backend/secret management.

• Store Scopus Author ID in faculty profile.

• Fetch permitted publication metadata.

• Create staging records.

• Match records to faculty.

• Run duplicate and validation checks.

• Send records to verification.

• Log API status, counts, errors and timestamps.

• Respect licensing terms and API rate limits.

• Do not use unauthorized automated Scopus website scraping.

12. Data Processing Engine

```
External Source → Fetch → Staging → Normalize → Faculty Match → Duplicate Check → Validation → Verification → Master Record
```

Staging: retain source, external ID, raw values, retrieval time, processing status and errors.

Normalization: standardize DOI, dates, titles, author strings, ISSN and publisher/journal values while retaining original source data where necessary.

13. Faculty / Author Matching

Priority: Scopus Author ID/ORCID exact match → DOI/external ID + author relationship → author name + affiliation + year → controlled similarity. Low-confidence matches go to review.

Every automatic match should store match confidence and match reason so reviewers can understand the decision.

14. Duplicate Detection

• Exact DOI = strong duplicate.

• Same external publication ID = strong duplicate.

• Normalized title + overlapping authors + same/similar year = strong candidate.

• Title similarity alone = possible duplicate only.

• Reviewer can merge, keep separate or dismiss.

• Every merge decision is audited.

15. Verification Workflow

```
Imported → Pending Verification → Under Review → Verified / Rejected / Needs Correction / Merged
```

• Reviewer sees source and imported metadata.

• System shows faculty match confidence.

• Potential duplicate candidates are shown.

• Evidence can be inspected.

• Reviewer approves, rejects, requests correction or merges.

• System stores reviewer, action, comments and timestamp.

• Only verified records are eligible for controlled official reports.

16. Evidence Management

• Upload supporting documents.

• Associate evidence with research records.

• Associate evidence with accreditation requirements.

• Store file metadata, uploader, timestamp and version.

• Restrict access by role.

• Show missing evidence before report generation.

• Maintain traceability to exact record/criterion.

17. NAAC Reporting Engine

The NAAC engine is configured from the applicable official NAAC framework/version. It uses the master database rather than maintaining a second research database.

```
Select NAAC version → Select scope → Identify metrics → Fetch verified data → Validate → Evidence →
Calculations → Output → PDF/Excel
```

• Criterion/metric configuration.

• Required-field mapping.

• Completeness validation.

• Evidence requirements.

• Configured calculations.

• Report table/output mapping.

• Report generation history.

18. NBA Reporting Engine

NBA is configured independently because its requirements and program-specific reporting structure differ from NAAC.

```
Select NBA version/program → Select scope → Identify requirements → Fetch verified data → Validate →
Evidence → Calculations → SAR/report output
```

• Program/scope configuration.

• Criterion/subcriterion mapping.

• Faculty/research field mapping.

• Evidence requirements.

• Calculated indicators where configured.

• SAR/report output mapping.

• Versioned report history.

19. Accreditation Requirement Matrix

Field	Purpose
Framework	NAAC or NBA
Version	Applicable official manual/framework version
Program / scope	Institution, department or NBA program
Criterion	Criterion identifier/name
Subcriterion / metric	Detailed requirement
Required data	Fields required
Master source	Database entity/field
Evidence	Supporting document
Calculation	Formula/derived value if applicable
Output	Table, metric, narrative or report item
Validation	Completeness and quality rule

20. Report Generator

• Select report type.

• Select NAAC/NBA framework and version where applicable.

• Select faculty, department, program or institution scope.

• Select academic/reporting year.

• Run validation.

• Display missing/invalid data.

• Preview output.

• Generate PDF/Excel.

• Store report metadata, framework version, scope and timestamp.

• Allow authorized export.

21. Dashboard & Analytics

• Total faculty and active faculty.

• ORCID-linked faculty.

• Scopus-linked faculty.

• Verified publications.

• Publications by year.

• Department-wise research.

• Patents.

• Projects and funding.

• Citations/metrics where available.

• Pending verification.

• Data completeness.

• Evidence completeness.

• Synchronization status and errors.

22. Automatic Synchronization

After the core system and integrations are stable, synchronization can run as a background job.

```
Scheduler → Source API → Fetch → Staging → Process → Match → Duplicate Check → Verification → Master DB
```

• Manual Sync option for authorized admins.

• Scheduled/background synchronization.

• Last successful sync per source.

• Counts for fetched/new/updated/duplicate/error records.

• Retry/backoff for temporary failures.

• Rate-limit handling.

• Complete sync history.

• No silent overwrite of verified records.

23. API Structure

• Authentication APIs.

• Faculty and department APIs.

• Research CRUD APIs.

• Import APIs.

• ORCID connector/service.

• Scopus/Elsevier connector/service.

• Matching service.

• Duplicate service.

• Verification APIs.

• Evidence APIs.

• Accreditation configuration APIs.

• Report APIs.

• Dashboard/analytics APIs.

• Audit APIs.

24. Recommended Technology Stack

Layer	Technology
Frontend	Next.js / React
UI	Tailwind CSS
Backend	Node.js + NestJS
Database	PostgreSQL
ORM	Prisma
Auth	JWT + RBAC; institutional SSO later if required
Background jobs	Redis + BullMQ
File storage	S3-compatible object storage
Reports	Server-side PDF/report generation
Deployment	Docker + cloud/VPS
Version control	Git + GitHub

25. Frontend Screen Plan

• Login.

• Dashboard.

• Faculty Management.

• Faculty Profile.

• Department Management.

• Research Management.

• Publication/Conference/Book/Patent/Project detail pages.

• Import Center.

• Synchronization Center.

• Verification Queue.

• Evidence Center.

• NAAC Report Center.

• NBA Report Center.

• Analytics.

• User & Role Management.

• Audit Logs.

• System Settings.

26. Security Requirements

• Secure password hashing.

• Server-side authorization.

• Role and scope checks on protected APIs.

• HTTPS in production.

• API secrets never exposed to frontend.

• Protected evidence storage.

• Input validation and sanitization.

• Audit logging.

• Database backup and recovery.

• Controlled report access.

• Framework/report version traceability.

27. Error Handling

• Invalid identifier → validation message.

• External API unavailable → failed/partial sync + retry.

• Rate limit → controlled retry/backoff.

• Invalid import row → row-level error.

• Possible duplicate → verification queue.

• Missing accreditation field → report validation error.

• Missing evidence → warning/block according to configuration.

• Unexpected error → technical log + safe user message.

28. Testing Plan

• Unit tests for business rules.

• API and database tests.

• Import/data-quality tests.

• Authorized ORCID/Scopus connector tests.

• Faculty matching tests.

• Duplicate detection tests.

• Verification workflow tests.

• Role/permission tests.

• Evidence access tests.

• Report calculation/output tests.

• Performance tests.

• Security tests.

• User acceptance testing.

29. Acceptance Criteria

• Faculty can be created/imported and uniquely identified.

• ORCID and Scopus IDs can be stored and managed.

• Research records can be created/imported.

• External data enters staging before approval.

• Matching is explainable.

• Duplicates are detected and reviewed.

• Records can be approved/rejected/merged.

• Evidence can be securely attached.

• NAAC and NBA can be configured independently.

• Reports use verified master data.

• Dashboards use approved data.

• Important actions appear in audit logs.

• Missing data is visible before report generation.

30. Exact Development Order

#	Stage	Output
1	Requirements	Freeze functional, data, user, evidence and reporting requirements.
2	NAAC/NBA mapping	Build official, versioned requirement matrices.
3	Architecture	Finalize application, database, integration and security boundaries.
4	ER/database	Build master research and operational schema.
5	Authentication/RBAC	Secure application and permissions.
6	Faculty/departments	Build institutional master data.
7	Research CRUD	Build research categories and forms.
8	Faculty profile	Create consolidated research profile.
9	Import engine	Support legacy Excel/CSV/manual data.
10	ORCID	Build authorized ORCID integration.
11	Scopus	Build authorized Elsevier/Scopus integration after access confirmation.
12	Processing	Normalization, matching and duplicate detection.
13	Verification	Approval, rejection, correction and merge workflow.

14	Evidence	Secure document management.
15	NAAC engine	Configure applicable NAAC requirements.
16	NBA engine	Configure applicable NBA requirements.
17	Reports	PDF/Excel/report generation.
18	Dashboard	Analytics and monitoring.
19	Background sync	Scheduled synchronization.
20	Security/testing	Hardening, validation and acceptance testing.
21	Deployment	Controlled DSU rollout.

31. Documents to Prepare Before Coding

• Software Requirements Specification (SRS).

• System Architecture Diagram.

• ER Diagram / Database Schema.

• Data Dictionary.

• NAAC Requirement Mapping Matrix.

• NBA Requirement Mapping Matrix.

• API Specification.

• UI Wireframes and Screen Flow.

• Matching Rules.

• Duplicate Detection Rules.

• Verification Workflow.

• Security and Permission Matrix.

32. End-to-End Example

A DSU faculty member has an ORCID iD and Scopus Author ID. A new paper appears in an authorized scholarly source. The system retrieves permitted metadata, places it in staging, matches it to the faculty, normalizes DOI/title/authors, checks duplicates, validates required fields, and sends it to verification. The faculty/admin reviews it and approves it. The system stores one master publication record. That same record can later appear in the faculty profile, department dashboard, NAAC mapping and NBA mapping without re-entry.

Core workflow:

```
Collect once → Process once → Verify once → Store once → Reuse everywhere
```

33. Final System Structure

• Presentation Layer — web interface and dashboards.

• Application Layer — business logic and workflows.

• Integration Layer — ORCID, authorized Scopus/Elsevier and imports.

• Processing Layer — normalization, matching and duplicate detection.

• Master Data Layer — PostgreSQL research database.

• Evidence Layer — secure document storage.

• Accreditation Layer — separate NAAC and NBA configurations.

• Reporting Layer — PDF/Excel/report generation.

• Security Layer — authentication, authorization, audit and backup.

34. Final Implementation Principle

The system must be built as a Faculty Research Intelligence Platform, not as a collection of separate accreditation forms. The research database is the core product; NAAC, NBA, dashboards and reports are controlled outputs of that database.

ONE MASTER RESEARCH DATABASE → NAAC + NBA + FACULTY REPORTS + DEPARTMENT REPORTS + DASHBOARDS

This design allows DSU to add research sources, update accreditation frameworks, add departments and introduce future analytics without rebuilding the core research system.